

Java Training

Interface, Polymorphism

Interface

■ Interface

● Interface là gì

Các phần, tiếp điểm, external spec... (môi trường của mọi vật...)

Là phần đã xác định external spec của method mà object đảm nhận 1 chức năng nào đó cần có

Là nội dung thể hiện những qui ước cần quyết định sẵn giữa người sử dụng và người cung cấp

● Khi biểu hiện interface bằng Java...

Là phần định nghĩa tổng hợp hằng số và khuôn mẫu của method cần thiết cho chức năng đặc định nào đó

Kiểu của giá trị trả về method name (argument list) ;

Định nghĩa interface

■ Định nghĩa của interface

● Cú pháp cơ bản

Có thể thực hiện nhiều chỉ định, ngăn cách bằng 「,」 (đa kế thừa)

```
public interface interface name extends list interface name muốn kế thừa {  
    // định nghĩa hằng số  
    Kiểu data tên biến = giá trị;  
    :  
    // định nghĩa khuôn mẫu của method  
    Kiểu giá trị trả về method name (list đối số);  
    :  
}
```

toàn bộ Field sẽ là 「public static final」

toàn bộ method là 「public abstract」

● Ví dụ (tạo thành PrintableInfo.java. Nếu compile thì PrintableInfo.class sẽ được tạo ra)

```
public interface PrintableInfo {  
    public void printInfo();  
}
```

Định nghĩa class implement interface

■ Định nghĩa class implement interface

● Cú pháp cơ bản

Trường hợp implement interface thì chỉ định là 「implements interface name」

```
public class class name extends super class name implements interface name list {  
  
    // bao gồm cả việc implement method đã được định nghĩa ở interface  
}
```

Mô tả việc implement toàn bộ method đã được liệt kê ở interface (nếu thiếu sẽ bị compile error)

● Ví dụ

```
public class Employee implements PrintableInfo {  
    (lược)  
    public void printInfo() {  
        // mô tả xử lý cụ thể để hiển thị giá trị thuộc tính của Employee  
    }  
}
```

Interface và kiểu data

■ Interface và kiểu data

● Cú pháp cơ bản

Kiểu data tên biến = new tên class implement interface(argument list);

Class name hoặc interface name

Class name

● Ví dụ: Trường hợp lấy class làm kiểu data

```
Employee emp = new Employee();  
emp.printInfo();
```

Optional method đã được định nghĩa ở Employee có thể gọi ra !

● Ví dụ: trường hợp lấy interface làm kiểu data

```
PrintableInfo obj1 = new Employee();  
obj1.printInfo();
```

Chỉ method đã được liệt kê ở interface là có thể gọi ra !

Cast kiểu tham chiếu (1)

■ Cast kiểu tham chiếu **Sub-class $\Leftrightarrow$ super class**

● Cú pháp cơ bản (giống với cast kiểu data cơ bản)

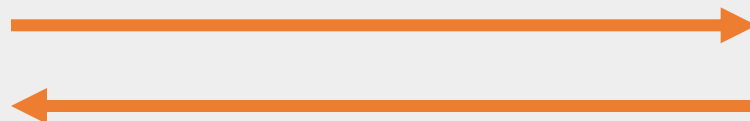
(kiểu data mới) giá trị hoặc biểu thức

Cú pháp thì giống, nhưng do ý nghĩa khác với trường hợp kiểu data cơ bản, nên cần phải chú ý !

● Point

Trường hợp không có điều kiện thì là ○
Có thể convert ngầm

**Sub-class
(Sales)**



**Super class (
Employee)**

Về nguyên tắc thì ×, nhưng tùy
trường hợp mà sẽ là ○
Cần convert kiểu tường minh

Cast kiểu tham chiếu (2)

■ Cast kiểu tham chiếu (sub-class $\Rightarrow$ super class)

Có thể convert ngầm
(Sales $\rightarrow$ Employee)

● Sales sa = new Sales();

● Employee emp = new Sales();

Object

Employee

Sales sa

Employee emp

Thuộc tính :

empld

987654

empName

田中花子

Tính chất đã kế thừa từ Employee

Thao tác : printInfo():void

Thuộc tính :

customerCount

5

Tính chất độc lập của Sales

Thao tác :

Sales(id:String, ename:String, cnt:int)

addCount():void

getCount():int

Có thể access vào thuộc tính • thao tác tùy ý mà object của Sales class có !

Tính chất đã kế thừa từ Employee

Thuộc tính :

empld

987654

empName

田中花子

Thao tác : printInfo():void

Thuộc tính :

customerCount

5

Tính chất độc lập của Sales

Thao tác :

Sales(id:String, ename:String, cnt:int)

addCount():void

getCount():int

Chỉ có thể access vào thuộc tính • thao tác đã kế thừa từ Employee class !

Cast kiểu tham chiếu (3)

■ Cast kiểu tham chiếu (super class $\Rightarrow$ sub-class)

Không thể convert ngầm !
(Employee $\rightarrow$ Sales)

● Employee emp = new Employee();

● Sales sa = new Employee (❌❌)

Employee Object

emp

Tính chất đã
định nghĩa ở
Employee

Thuộc tính :

empld 987654

empName 田中花子

Thao tác :

printlnInfo():void

Sales

sa

Tính chất đã
định nghĩa ở
Employee

Thuộc tính :

empld 987654

empName 田中花子

Thao tác : printlnInfo():void

Không tồn tại tính chất cố hữu ở
Sales object !

Thuộc tính :

customerCount

Thao tác :

Sales(id:String, ename:String, cnt:int)

addCount():void

getCount():int

Do đang bị thiếu nội dung cần thiết cho object
của Sales class nên bị compile error !

Cast kiểu tham chiếu (4)

■ Cast kiểu tham chiếu (sub-class ⇒ super class ⇒ sub-class)

```
Sales sa = new Sales();
Employee emp = sa;
Sales sa2 = (Sales)emp; // không thể convert ngầm
```

Có trường hợp có thể một cách mang tính giới hạn !

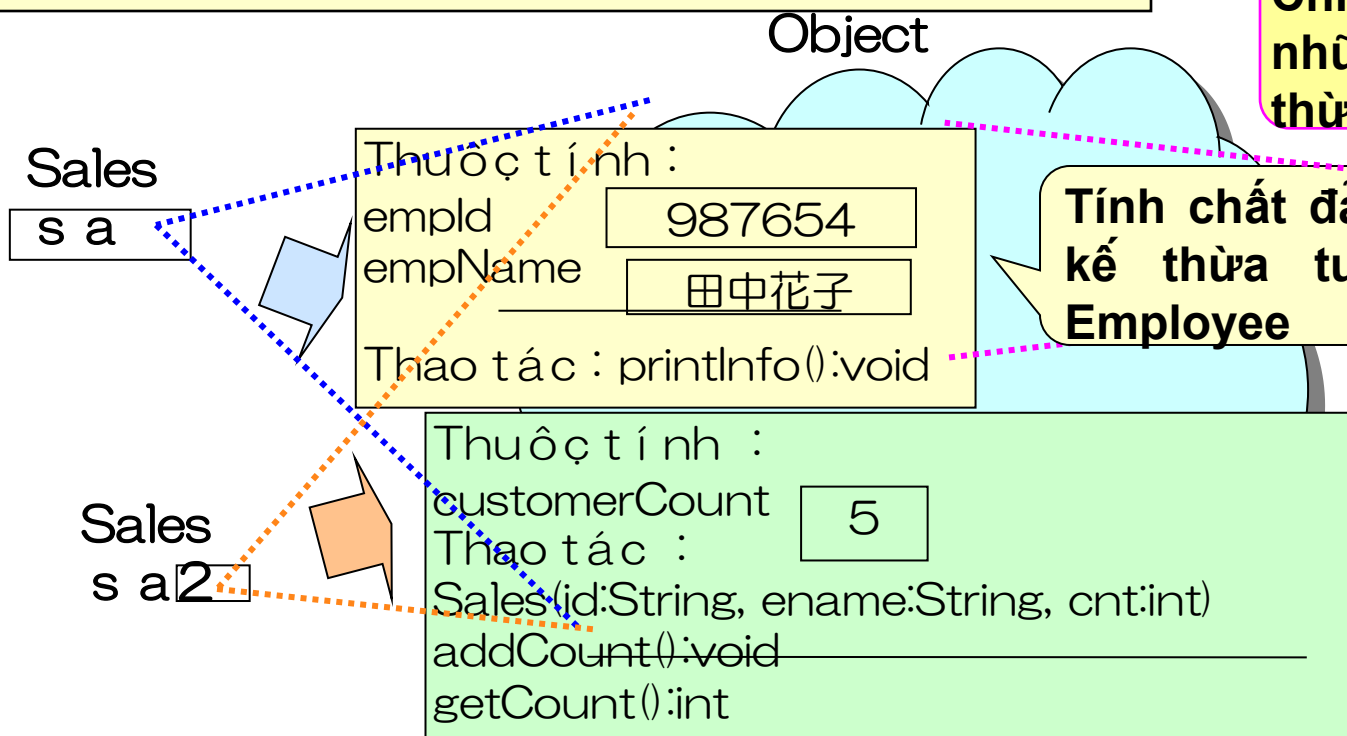
Chỉ có thể access vào những tính chất đã kế thừa từ super class !

Tính chất đã kế thừa từ Employee

Employee

emp

Tính chất độc lập của Sales



Do đang có những thứ cần thiết cho object của Sales class nên nếu là convert rõ ràng thì có thể thực hiện ! Có thể access vào thuộc tính • thao tác tùy ý mà object của Sales class có !

Polymorphism(1)

■ Polymorphism

● Polymorphism

Hoạt động của một object không phải là đặc tính xác định bởi phía yêu cầu xử lý mà phải là đặc tính xác định dựa vào object đã tiếp nhận yêu cầu.

Lấy nhiều object của class cùng loại làm đối tượng, có thể yêu cầu xử lý thống nhất !

● Nếu biểu hiện polymorphism bằng Java...

Nếu sử dụng biến số lấy super class hoặc interface làm kiểu data để gọi method, thì từng object có biến đang tham chiếu phải thực hiện hành vi đã được định nghĩa trong chính object đó.

Các object đã nhận yêu cầu hoạt động độc lập !

Polymorphism (2)

■ Cast và override kiểu tham chiếu

Chỉ có thể truy cập từ thuộc tính • thao tác đã kế thừa từ Employee class !
(Loại trừ method đã được override)

Trường hợp method đang được override, việc implement của sub class sẽ được gọi !

```
Employee emp = new Sales();
```

```
// Có thể gọi ra method đã được định nghĩa bởi Employee  
System.out.println(emp.getEmpName());
```

```
// Không thể gọi ra method đã được định nghĩa bởi Sales  
System.out.println(emp.getCount()); ←Error
```

```
// Có thể gọi ra method đã được override  
emp.printInfo(); ←việc implement của Sales được gọi ra !
```

Employee

emp

Tính chất đã kế thừa từ Employee

Thuộc tính :

empId 987654

empName 田中花子

Thao tác getEmpName():String
printInfo():void

Tính chất của riêng Sales

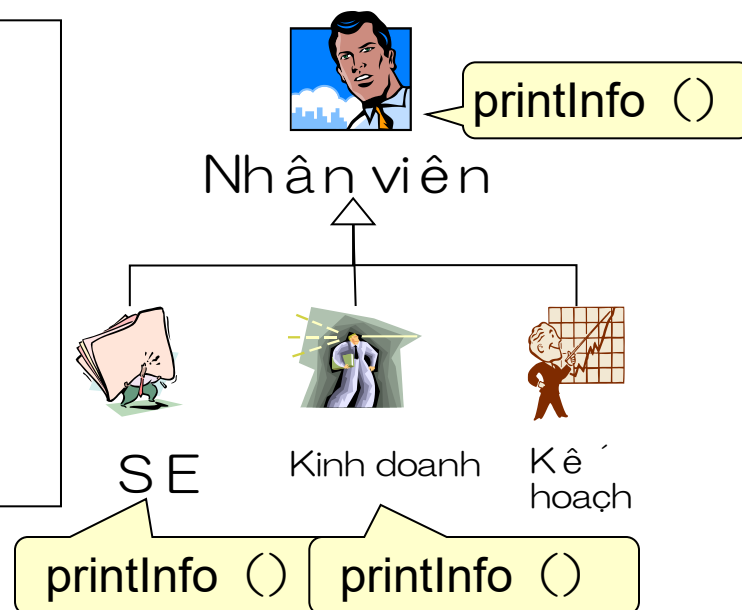
Thuộc tính :
customerCount 5

Thao tác :
Sales(id:String, ename:String, cnt:int)
addCount():void
getCount():int
printInfo():void

Polymorphism (3)

■ Ví dụ implement trong Java

```
Employee[] empArray = new Employee[3];
empArray[0] = new Sales("100200", "田中花子", 5);
empArray[1] = new SE("300400", "松本一郎", "DB");
empArray[2] = new Sales("500600", "田中裕樹", 8);
for (int i = 0; i < empArray.length; i++) {
    System.out.println("-----");
    empArray[i].printInfo();
}
```



 empId:100200
 empName:田中花子
 customerCount:5

printInfo method của
Sales

 empId:300400
 empName:松本一郎
 specializedField:DB

printInfo method của
SE

 empId:500600
 empName:田中裕樹
 customerCount:8

printInfo method của
Sales